5.2 Горутины

Горутины — это легковесные потоки для параллельного выполнения задач, которые легко создавать с помощью ключевого слова `go`

Для визуализации можно использовать диаграмму, показывающую несколько горутин, работающих параллельно.

Мы также упомянем, что горутины очень дешевы в создании, что позволяет запускать тысячи или даже миллионы из них без значительных накладных расходов, что подтверждается исследованиями на платформах, таких как Golang Docs.

Демонстрация создания горутины

Простой пример:

```
package main
import "fmt"
func hello() {
  fmt.Println("Hello from goroutine")
}

func main() {
  go hello()
  fmt.Println("main function")
}
```

`main` завершается раньше, чем горутина успевает выполниться.

Добавим `time.Sleep(1 * time.Second)` для исправления:

```
package main

import (
  "fmt"
  "time"
)

func hello() {
  fmt.Println("Hello from goroutine")
}

func main() {
  go hello()
  time.Sleep(1 * time.Second)
  fmt.Println("main function")
}
```

Теперь выводится и "Hello from goroutine".

`time.Sleep` — это временное решение, и в реальных приложениях лучше не использовать такой подход

Множественные горутины

Запустим несколько горутин одновременно с печатью чисел и букв:

```
package main

import (
  "fmt"
  "time"
)

func numbers() {
   for i := 1; i <= 5; i++ {
   time.Sleep(250 * time.Millisecond)
   fmt.Printf("%d ", i)
 }
}

func alphabets() {
   for i := 'a'; i <= 'e'; i++ {
   time.Sleep(400 * time.Millisecond)
   fmt.Printf("%c ", i)
 }
}

func main() {
   go numbers()
   go alphabets()
   time.Sleep(3 * time.Second)
   fmt.Println("main terminated")
}
```

Материалы

- Golang Docs Goroutines in Golang
- Golangbot Goroutines